

RageJEX EpicTask Dashboard – Product Specification Document (PRD)

Project Overview

Product Name

EpicTask Dashboard

Brand Name

RageJEX

Tagline

Plan. Track. Deliver.

Project Type

Component-Based React.js SaaS Dashboard

Objective

Build a modern project management dashboard inspired by ClickUp that helps teams manage projects, tasks, dependencies, workloads, notifications, and schedules. The application must satisfy all case-study requirements while maintaining a professional SaaS-style user experience.

Technology Stack

Frontend

- React.js
- React Router DOM

- Tailwind CSS
- Lucide React Icons
- Recharts

Storage

- LocalStorage

Authentication

- LocalStorage-based Login & Signup

State Management

- Context API + Custom Hooks
-

Folder Structure

Project Folder Structure

```
src/  
├── assets/  
├── components/  
│   ├── common/  
│   ├── layout/  
│   ├── dashboard/  
│   ├── projects/  
│   ├── tasks/  
│   ├── dependencies/  
│   ├── notifications/  
│   └── workload/  
├── pages/  
├── routes/  
├── context/  
├── hooks/  
├── services/  
├── utils/  
├── data/  
├── App.jsx  
└── main.jsx
```

User Roles

Admin User

Single-user system.

Capabilities:

- Create Projects
 - Manage Tasks
 - Manage Dependencies
 - Manage Team Members
 - View Analytics
 - Manage Notifications
 - Track Critical Path
-

Application Modules

Authentication Module

Login

Fields:

- Email
- Password

Validation:

- Required Fields
- Valid Credentials

Signup

Fields:

- Full Name
- Email
- Password
- Confirm Password

Validation:

- Required Fields
- Email Format
- Password Length ≥ 8
- Password Match

Protected Routes

Authenticated users only:

- Dashboard
 - Projects
 - Tasks
 - Dependencies
 - Notifications
 - Workload
 - Settings
-

Dashboard Module

Purpose:

Provide a complete overview of project activity.

Widgets:

KPI Cards

- Total Projects
- Total Tasks
- Active Tasks
- Blocked Tasks
- Overdue Tasks

Charts

- Task Status Distribution
- Team Workload Distribution
- Deadline Risk Analysis

Activity Feed

Recent Actions:

- Project Created
 - Task Updated
 - Dependency Added
 - Deadline Changed
-

Projects Module

Purpose:

Manage company projects.

Features:

- Create Project
- Edit Project
- Delete Project
- View Project

Project Model:

```
{  
  id,  
  name,  
  description,  
  owner,  
  startDate,  
  endDate,  
  status  
}
```

Status:

- Planning
 - Active
 - Completed
 - On Hold
-

Task Module

Purpose:

Manage project tasks.

Features:

- Add Task
- Edit Task
- Delete Task
- Search Tasks
- Filter Tasks
- Sort Tasks

Task Model:

```
{  
  id,  
  title,  
  projectId,  
  assignee,  
  duration,  
  status,  
  priority,  
  deadline,  
  delayDays,  
  blockedBy  
}
```

Status:

- Todo
- In Progress
- Completed
- Blocked

Priority:

- Low
 - Medium
 - High
 - Critical
-

Task Folder Viewer

Case Study Requirement

Purpose:

Display project hierarchy and assigned ownership.

Structure:

Project

└ Task

└ Task

└ Task

Information:

- Task Name
 - Assignee
 - Status
 - Deadline
-

Schedule Undo/Redo Module

Case Study Requirement

Purpose:

Track task duration changes.

Data Structure:

- Past Stack
- Current State
- Future Stack

Functions:

- Undo Change
- Redo Change

Change History:

- Previous Duration

- New Duration
 - Timestamp
-

Alert Organizer Module

Case Study Requirement

Purpose:

Manage notification queue.

Notification Model:

```
{  
  id,  
  type,  
  message,  
  timestamp  
}
```

Queue Logic:

FIFO

Examples:

- Task Assigned
 - Deadline Updated
 - Project Created
-

Template Checker Module

Case Study Requirement

Purpose:

Validate project creation against company standards.

Required Template Fields:

- Project Name
- Team Lead
- Milestones
- Start Date
- End Date

Validation Result:

- Approved
 - Rejected
-

Deadline Risk Sorter

Case Study Requirement

Purpose:

Identify risky tasks.

Risk Formula:

Risk Score = Delay Days

Sorting:

Highest Risk → Lowest Risk

Output:

Top Delayed Tasks

Task Lock Hub

Case Study Requirement

Purpose:

Track blocked tasks.

Example:

Design

↓

Frontend

↓

Testing

↓

Deployment

Display:

- Blocking Task
 - Waiting Task
 - Status
-

Quickest Project Completion Finder

Case Study Requirement

Purpose:

Display Critical Path.

Input:

Task Dependencies

Output:

Critical Path Chain

Example:

Design

→ Frontend

→ Testing

→ Deployment

Total Duration:

18 Days

Team Workload Assignor

Case Study Requirement

Purpose:

Distribute work fairly.

Member Model:

```
{  
  id,  
  name,  
  assignedTasks  
}
```

Logic:

Assign task to member with minimum workload.

Output:

Suggested Assignee

Notifications Module

Features:

- Notification Queue
- Notification History
- Notification Types

Categories:

- Success
 - Warning
 - Info
 - Error
-

Workload Module

Features:

- Team Cards
- Workload Charts
- Auto Assignment

Charts:

- Tasks Per Member
 - Workload Comparison
-

Settings Module

Features:

- Theme Selection
- LocalStorage Reset
- Profile Information

Theme Options:

- Light
 - Dark
 - System
-

LocalStorage Schema

users

currentUser

projects

tasks

members

notifications

history

activities

settings

Demo Seed Data Requirements

Application must load with realistic sample data on first launch.

Minimum Seed Data:

Projects:

- 3 Sample Projects

Tasks:

- 12 Sample Tasks

Team Members:

- 5 Sample Members

Dependencies:

- Multiple Linked Tasks

Notifications:

- Sample Notification Queue

Activities:

- Sample Activity Feed

Purpose:

Prevent empty dashboard states and improve user experience during demonstration.

Routing Structure

/auth/login

/auth/signup

/dashboard

/projects

/tasks

/dependencies

/notifications

/workload

/settings

UI Design Requirements

Design Style:

ClickUp Inspired SaaS Dashboard

Visual Characteristics:

- Modern
- Colorful
- Clean
- Responsive
- Professional

Primary Color:

#7C3AED

Secondary Color:

#3B82F6

Success:

#22C55E

Warning:

#F59E0B

Danger:

#EF4444

Background:

#F8FAFC

Responsiveness Requirements

Desktop:

Full Dashboard Layout

Tablet:

Collapsible Sidebar

Mobile:

Drawer Navigation

Responsive Cards

Responsive Tables

Non-Functional Requirements

- Component-Based Architecture
 - Reusable Components
 - Clean Folder Structure
 - Responsive Design
 - LocalStorage Persistence
 - Modern UI/UX
 - Accessible Forms
 - Optimized Rendering
-

Development Standards

Coding Requirements:

- Functional Components Only
- React Hooks Only
- No Class Components
- Component-Based Architecture
- Reusable Components
- Tailwind CSS Only
- No Inline Styling
- Responsive First Design
- Clean Folder Structure
- Meaningful Component Names
- Clean State Management
- LocalStorage Persistence
- Accessible Forms
- Maintainable Code

Performance Requirements:

- Avoid Unnecessary Re-renders
- Reusable Utility Functions
- Modular Architecture

Features Not Included

- Backend
- Database
- JWT Authentication

- APIs
- Redux
- Multi-user Collaboration
- Cloud Deployment Logic